

OSSP-WEEK2-PRACTICAL

Task1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source code:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    printf("Source file opened successfully.\n");
    printf("Destination file opened successfully.\n");

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            if (bytesWritten != bytesRead) {
                perror("Error writing to destination file");
                close(source);
                close(destination);
                return 1;
            }
        }

        if (bytesRead < 0) {
            perror("Error reading source file");
        } else {
            printf("File contents copied successfully.\n");
        }

        close(source);
        close(destination);

        printf("Files closed successfully.\n");

        return 0;
    }
}
```

Output:

```
fork-example.c.save      task3.c
sainath@LAPTOP-GJ94VPOB:~$ ./week2_task1
Source file opened successfully.
Destination file opened successfully.
File contents copied successfully.
Files closed successfully.
sainath@LAPTOP-GJ94VPOB:~$ |
```

Observation: I learned how to use the Linux system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. I understood how data is read from the source file into a buffer and then written to the destination file. I also learned how control moves from user space to kernel space when a system call is made, and how the operating system manages file resources during the file-copy operation.

Task2: Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Command: `strace cat sample.txt`

Output:

```
+++ exited with 0 +++
sainath@LAPTOP-GJ94VPOB:~$ strace -e trace=execve,openat,read,write,fstat,mmap,munmap,close,exit_group cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7fff8d4ee848 /* 26 vars */) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x79900cca1000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20387, ...}) = 0
mmap(NULL, 20387, PROT_READ, MAP_PRIVATE, 3, 0) = 0x79900cc9c000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x79900ca00000
mmap(0x79900ca28000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x79900ca28000
mmap(0x79900cbb0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x79900cbb0000
mmap(0x79900cbff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x79900cbff000
mmap(0x79900cc05000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_
```

Observation: I learned how to use the `strace` utility to trace the system calls generated by the `cat` command. I learned how `execve()` starts the program, `openat()` opens the file, `read()` reads the file contents, and `write()` displays them on the terminal. I also learned how `fstat()`, `mmap()`, `munmap()`, `close()`, and `exit_group()` support file access, memory management, resource cleanup, and process termination.